# Approximate Multiply–Accumulate Units for Energy-Efficient DNN Accelerators

Error Characterization and Application-Level Evaluation

Sina Mozaffari Rad
Fatemeh Dastyafteh

Co-developed project | https://github.com/GitCodingMates/approx-mac-dnn

November 2026

**Abstract**

Multiply–accumulate (MAC) operations dominate both the computation and the energy budget of deep neural network (DNN) inference. Because DNNs are inherently tolerant to small numerical errors, the exact multiplier inside each MAC can be replaced with a cheaper *approximate* multiplier. This project studies that trade-off for 8-bit signed arithmetic. Bit-accurate models of an exact array multiplier, three column-truncated multipliers, and Mitchell's logarithmic multiplier were developed and characterized exhaustively over all $2^{16}$ input pairs. A fully-connected network was trained on MNIST, quantized to int8 with int32 accumulation, and evaluated with every multiplication routed through each approximate design. Results show that even a multiplier with a mean relative error of 23.9 % reduces classification accuracy by only 0.42 %. All designs were implemented in Verilog, verified exhaustively against the reference models, and synthesized with Yosys. The truncated multiplier reduces multiplier area by 59.8 % and complete MAC area by 38.9 %, whereas the logarithmic multiplier saves less than 5 % at this operand width, showing that the best approximation strategy depends on the datapath width and cannot be chosen from error metrics alone.

## 1 Introduction

The processing elements (PEs) of a DNN accelerator are, at their core, MAC units. In an 8-bit datapath the multiplier is the largest and most power-hungry part of the PE, so its cost scales directly with the size of the PE array. *Approximate computing* exploits the error resilience of DNNs: errors in individual products are partially averaged out across long dot products, and the final classification decision (an `argmax`) is insensitive to small perturbations of the output scores.

The goals of this project are:

1. to build bit-accurate, hardware-oriented models of representative approximate multipliers;
2. to characterize their arithmetic error with standard metrics;
3. to measure their impact on the accuracy of a quantized neural network;
4. to implement them in synthesizable Verilog and verify the RTL exhaustively against the reference models;
5. to quantify the hardware savings by logic synthesis and identify the best area–accuracy trade-offs (Section 7).

## 2 Multiplier Designs

All designs operate on 8-bit two's-complement operands $a, b \in [-128, 127]$ and are implemented in sign-magnitude form: the magnitudes $|a|, |b| \in [0, 128]$ are multiplied and the sign is restored as $\mathrm{sgn}(a) \oplus \mathrm{sgn}(b)$. This makes the error of every approximate design symmetric around zero.

### 2.1 Exact array multiplier (baseline)

The product is the sum of $N^2$ partial-product bits,

$$P = \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} a_i \, b_j \, 2^{i+j}, \qquad N = 8. \tag{1}$$

### 2.2 Column-truncated multiplier

All partial-product bits whose weight falls below column $k$ are removed:

$$P_{\text{trunc}}(k) = \sum_{i+j \,\geq\, k} a_i \, b_j \, 2^{i+j}. \tag{2}$$

Each removed bit eliminates one AND gate and its share of the carry-save adder tree. For $k = 4, 6, 8$, respectively 10, 21 and 36 of the 64 partial-product bits are removed. No error-compensation constant is added, so the design always under-estimates the product magnitude.

### 2.3 Mitchell logarithmic multiplier

Writing each operand as $x = 2^k(1 + f)$, where $k = \lfloor \log_2 x \rfloor$ is found by a leading-one detector (LOD) and $0 \leq f < 1$, Mitchell's algorithm [1] approximates $\log_2(ab) \approx k_1 + k_2 + f_1 + f_2$. With $x_i = |a_i| - 2^{k_i}$, an exact-integer form suitable for hardware is

$$S = x_1 2^{k_2} + x_2 2^{k_1}, \qquad P_{\text{Mitchell}} = \begin{cases} 2^{k_1+k_2} + S, & S < 2^{k_1+k_2} \\ 2S, & \text{otherwise.} \end{cases} \tag{3}$$

The datapath therefore consists only of two LODs, barrel shifters and a single adder; no partial-product array is required.

**Error bound.** The bound below was derived by Mitchell [1]; it is restated here in the notation of Eq. (3). Since $x_i = 2^{k_i} f_i$, we have $S = 2^{k_1+k_2}(f_1 + f_2)$, while the exact product is $P = 2^{k_1+k_2}(1 + f_1)(1 + f_2)$. Two cases follow.

*Case* $f_1 + f_2 < 1$. Here $P_{\text{Mitchell}} = 2^{k_1+k_2}(1 + f_1 + f_2)$ and

$$\frac{P - P_{\text{Mitchell}}}{P} = \frac{f_1 f_2}{(1 + f_1)(1 + f_2)} \;\geq 0. \tag{4}$$

*Case* $f_1 + f_2 \geq 1$. Here $P_{\text{Mitchell}} = 2^{k_1+k_2+1}(f_1 + f_2)$ and

$$\frac{P - P_{\text{Mitchell}}}{P} = \frac{(1 - f_1)(1 - f_2)}{(1 + f_1)(1 + f_2)} \;\geq 0. \tag{5}$$

Both expressions are non-negative, so the multiplier never over-estimates. In the first case, write $g(f) = f/(1 + f)$, which is increasing; the error is $g(f_1)g(f_2)$ and grows with both fractions, so its supremum lies on the boundary $f_1 + f_2 = 1$. In the second case, $h(f) = (1 - f)/(1 + f)$ is decreasing and the error $h(f_1)h(f_2)$ is also largest on $f_1 + f_2 = 1$. On that line, with $f_2 = 1 - f_1$, both cases give the same value,

$$E(f_1) = \frac{f_1(1 - f_1)}{(1 + f_1)(2 - f_1)}, \tag{6}$$

which is symmetric about $f_1 = \frac{1}{2}$. Setting $E'(f_1) = 0$ gives $(1-2f_1)\big[(1+f_1)(2-f_1) - f_1(1-f_1)\big] = 0$, where the bracket equals 2, so $f_1 = f_2 = \frac{1}{2}$ and

$$E_{\max} = \frac{\frac{1}{2} \cdot \frac{1}{2}}{\frac{3}{2} \cdot \frac{3}{2}} = \frac{1}{9} \approx 11.1\,\%. \tag{7}$$

This is the well-known worst-case relative error of the Mitchell multiplier [1, 2]. For 8-bit operands $f = \frac{1}{2}$ is reachable, e.g. $a = b = 3$ gives $P = 9$ and $P_{\text{Mitchell}} = 8$. The exhaustive evaluation of the bit-accurate model over all operand pairs (Section 4) confirms a maximum relative error of exactly $1/9$, attained whenever $|a| = 3 \cdot 2^i$ and $|b| = 3 \cdot 2^j$.

# 3 Methodology

## 3.1 Bit-accurate reference models

Each design is described in Python/NumPy at the bit level (`python/approx_mult.py`), mirroring the intended RTL structure (partial-product selection, LOD, shifts). These models serve as the golden reference for RTL verification. For fast evaluation, each model is expanded into a $256 \times 256$ product look-up table.

## 3.2 Error metrics

Every design is evaluated over all 65,536 input pairs. With $P$ the exact and $\hat{P}$ the approximate product, and $\mathrm{ED} = |\hat{P} - P|$, the following metrics are reported [2]:

- **ER** – error rate, fraction of inputs with ED $\neq 0$;
- **MED** – mean error distance, $\mathbb{E}[\mathrm{ED}]$;
- **NMED** – MED normalized by the maximum product ($128^2$);
- **MRED** – mean relative error distance, $\mathbb{E}[\mathrm{ED}/|P|]$ over $P \neq 0$;
- **Max ED** – worst-case absolute error.

## 3.3 Quantized neural network

A 784–128–10 multilayer perceptron with ReLU activation was trained on MNIST [5] for eight epochs using mini-batch SGD (batch 128, learning rate 0.1), implemented directly in NumPy. The trained network was then quantized following the integer-only inference scheme of [4]:

- inputs: $x_q = \mathrm{round}(127\,x)$, $x \in [0, 1]$;
- weights: symmetric per-tensor int8, $s_w = \max|W|/127$;
- biases: int32 at scale $s_x s_w$;
- accumulation: int32;
- hidden activations: requantized to unsigned 7-bit values using a scale calibrated on 5,000 training images (99.99th percentile).

During approximate inference, *every* multiplication in both layers is replaced by the corresponding approximate multiplier, exactly as it would occur inside the accelerator's PEs. Accuracy is measured on the full 10,000-image test set.

# 4 Results

## 4.1 Arithmetic error

Table 1 summarizes the exhaustive error characterization, and Figure 1 shows how the relative error is distributed over the input space.

Table 1: Error characterization over all $2^{16}$ signed 8-bit input pairs.

| Multiplier | ER (%) | MED | NMED (%) | MRED (%) | Max ED |
|---|---|---|---|---|---|
| Exact | 0.00 | 0.00 | 0.000 | 0.00 | 0 |
| Truncated $k$=4 | 81.25 | 12.25 | 0.075 | 1.72 | 49 |
| Truncated $k$=6 | 93.75 | 80.25 | 0.490 | 7.59 | 321 |
| Truncated $k$=8 | 98.05 | 384.75 | 2.348 | 23.87 | 1537 |
| Mitchell | 87.89 | 151.34 | 0.924 | 3.70 | 1024 |

The measured MRED of the Mitchell multiplier (3.70 %) agrees with the value reported in the literature (≈3.8 %), which validates the model. Truncation exhibits a clear knob: each increase of $k$ by two columns raises the MED by roughly a factor of five.

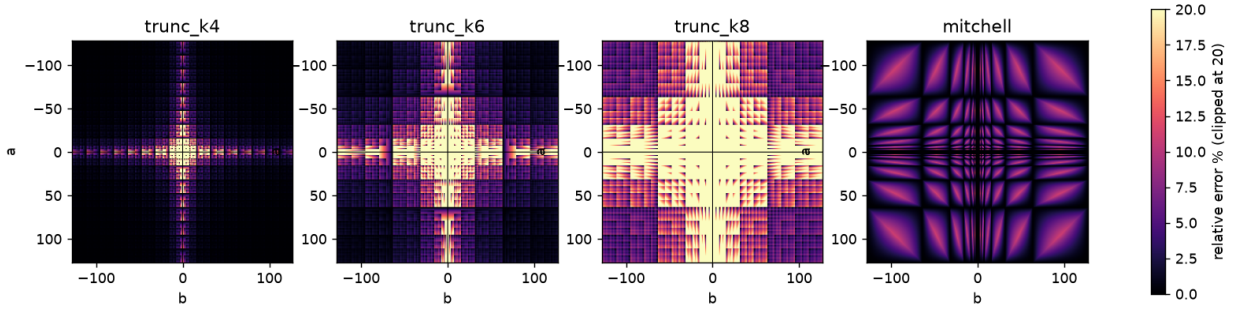



Figure 1: Relative error (%) of each approximate multiplier as a function of its operands. Truncation errors concentrate at small operands, where the removed low-order columns carry a larger share of the product; Mitchell's error depends on the fractional parts of both operands and is bounded at about 11 %.

### 4.2 Application-level accuracy

Table 2: MNIST test accuracy of the int8 MLP with each multiplier. Float32 reference: 96.82 %.

| Multiplier | Accuracy (%) | Δ vs. exact int8 |
|---|---|---|
| Exact (int8) | 96.81 | – |
| Truncated $k$=4 | 96.82 | +0.01 |
| Truncated $k$=6 | 96.77 | −0.04 |
| Truncated $k$=8 | 96.39 | −0.42 |
| Mitchell | 96.77 | −0.04 |

Table 2 shows the central result of this project: arithmetic error and application error are only weakly coupled. Int8 quantization itself costs just 0.01 %. Truncation up to $k = 6$ and the Mitchell multiplier are practically lossless, and even $k = 8$ truncation—which removes more than half of the partial-product array and has an MRED above 20 %—loses less than half a percentage point. Two effects explain this resilience:

1. **Error averaging.** Each output neuron accumulates up to 784 products; independent errors partially cancel, so the relative error of the sum is far smaller than that of a single product.
2. **Decision robustness.** Classification depends only on the ranking of the output scores, so a systematic, roughly proportional under-estimation (as produced by both truncation and Mitchell) rarely changes the `argmax`.

## 5 RTL Implementation

All designs are written in synthesizable Verilog-2001 (`rtl/`) and share a common `sign_mag` front end that converts the two's-complement operands to 8-bit magnitudes and computes the product sign.

- `mul_exact` – baseline $8 \times 8$ magnitude multiplier.
- `mul_trunc #(K)` – the partial-product array is built row by row; in row $j$ the bits of $|a|$ below position $K - j$ are masked with an elaboration-time constant, so the corresponding AND gates and adder inputs are removed by synthesis rather than gated at run time. A single parameter covers the whole design space.
- `mul_mitchell` – two priority-encoder LODs, mantissa extraction by clearing the leading one, two barrel shifters, one 16-bit adder and a comparator/multiplexer that selects between the two cases of Eq. (3).

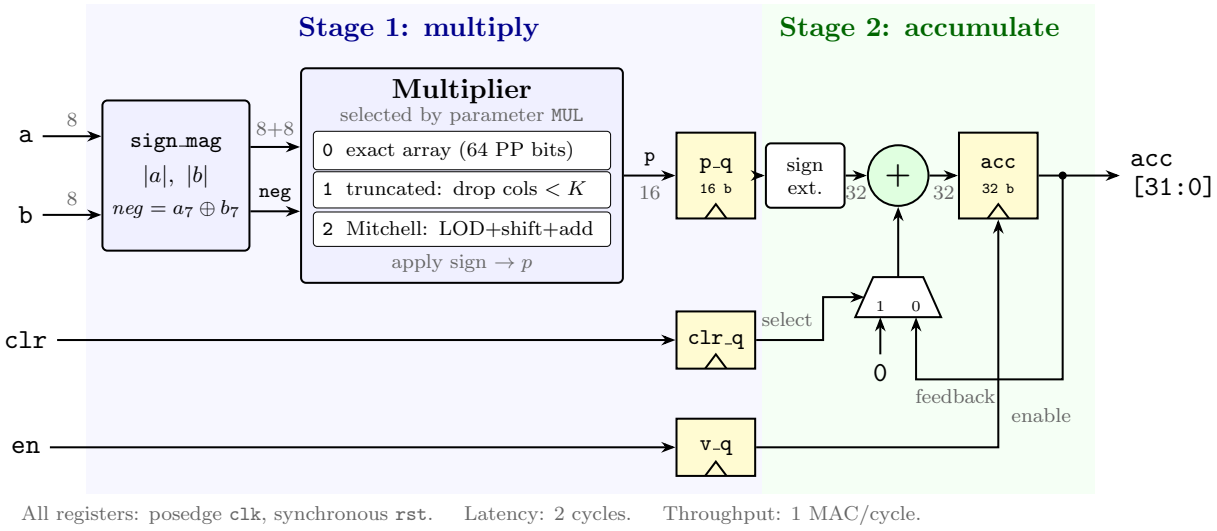



Figure 2: Block diagram of the pipelined MAC unit. Yellow boxes with a clock triangle are registers. Stage 1 converts the operands to sign-magnitude form and computes the product with the multiplier selected by `MUL`; stage 2 sign-extends the registered product and adds it to the 32-bit accumulator. The delayed `clr_q` selects 0 instead of the accumulator feedback, so a new dot product starts without an idle cycle, and the delayed `en` (`v_q`) enables the accumulator register.

- `mac #(MUL,K)` – a two-stage pipelined MAC. Stage 1 registers the product, stage 2 sign-extends it and accumulates into a 32-bit register. A `clr` input restarts the accumulation so consecutive dot products can be streamed without bubbles, and an `en` input supports idle cycles. The multiplier is selected at elaboration time with a `generate` block, so all variants share one PE description.

Figure 2 shows the resulting datapath. Only the multiplier block changes between the evaluated designs; the operand conversion, pipeline registers, adder and accumulator are identical. This is why the area saving of a complete MAC (Section 7) is smaller than that of the multiplier alone.

## 6 Functional Verification

Verification reuses the Python models as the golden reference. `python/gen_vectors.py` exports, for each design, the expected product of all $2^{16}$ operand pairs as a hex table indexed by $\{a, b\}$. Two self-checking testbenches (`tb/`) were run in ModelSim; both are also compatible with Icarus Verilog.

- `tb_mul` applies all 65,536 operand pairs to the five multiplier instances simultaneously and compares every output bit-exactly with the golden table.
- `tb_mac` streams 20,000 random operand pairs into three MACs (exact, truncated $K$=6, Mitchell) with randomly inserted idle cycles and a `clr` every 784 terms, mimicking the first layer of the MLP. A cycle-accurate reference accumulator with the same two-cycle latency checks the output on every cycle.

The exhaustive comparison proved its value during development: the first Mitchell implementation failed on 31,193 of the 65,536 pairs. The cause was the shift amount $k_1 + k_2$, which Verilog evaluated with the 3-bit width of its operands and therefore wrapped for $k_1 + k_2 \geq 8$. Widening the sum to 4 bits fixed the design. Such width-dependent errors appear only for specific operand ranges and are easily missed by random testing, but cannot escape an exhaustive check.

Table 3: RTL verification results.

| Testbench | Design | Checks | Mismatches |
|---|---|---|---|
| `tb_mul` | exact | 65,536 | 0 |
| | truncated $K$=4 | 65,536 | 0 |
| | truncated $K$=6 | 65,536 | 0 |
| | truncated $K$=8 | 65,536 | 0 |
| | Mitchell | 65,536 | 0 |
| `tb_mac` | exact, trunc. $K$=6, Mitchell | 20,005 cycles | 0 |

### 6.1 Simulation Environment and Code Coverage

Simulation is performed in ModelSim SE-64 2020.4. The `sim/modelsim` directory contains a ModelSim project (`approx_mac.mpf`) and a set of Tcl scripts: `compile.do` builds the `work` library, `sim_mul.do` and `sim_mac.do` run the two testbenches (with a predefined wave layout in GUI mode), and `regression.do` runs the complete suite in batch mode, merges the coverage databases of both tests and writes a coverage report. The RTL is compiled with statement, branch, condition, expression and toggle coverage enabled. Figure 3 shows the console output of the batch regression; the complete transcript is kept in `results/modelsim_regression.log`.

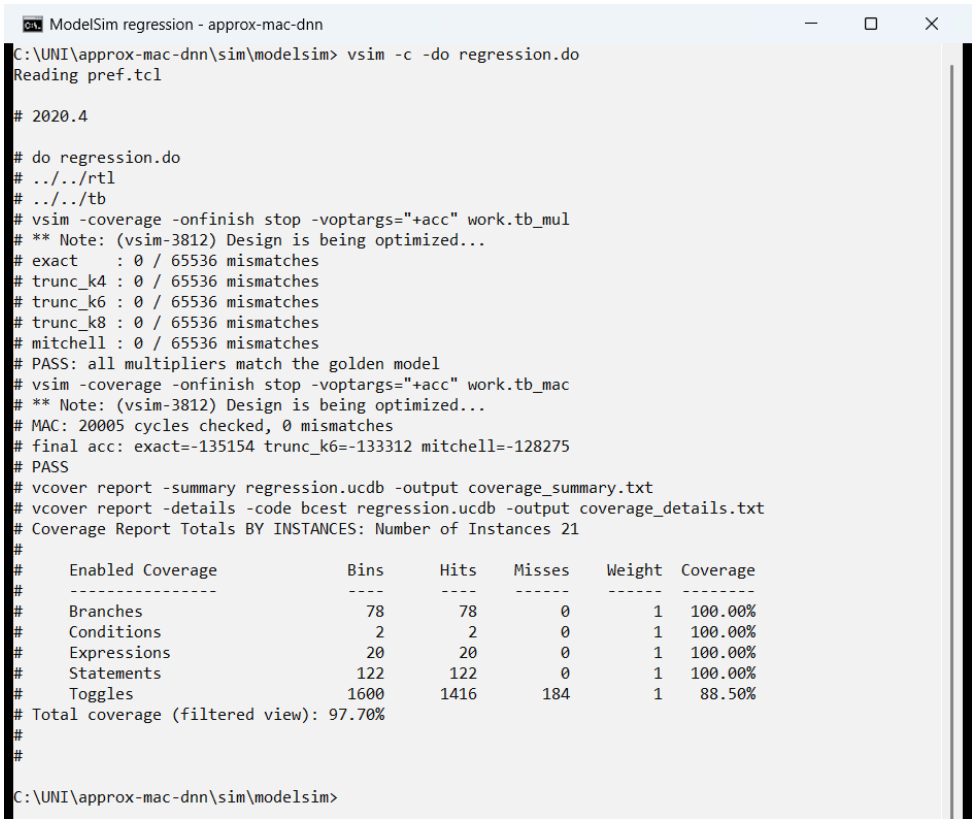



Figure 3: Batch regression in ModelSim (`vsim -c -do regression.do`): exhaustive multiplier test, cycle-accurate MAC test and merged coverage summary.

All statements, branches, conditions and expressions are exercised (Table 4). Every missed toggle bin was inspected and is structurally unreachable rather than a gap in the tests: the product bits below column $K$ of the truncated multipliers are constant zero by design, bit 15 of the product magnitude can never be set because the largest magnitude is $128 \times 128 = 2^{14}$, and the upper bits of the 32-bit accumulator are not reached by 784-term dot products of int8 data.

Table 4: Merged code coverage of the RTL over both testbenches (21 instances).

| Metric | Bins | Hit | Coverage (%) |
|---|---|---|---|
| Statements | 122 | 122 | 100.0 |
| Branches | 78 | 78 | 100.0 |
| Conditions | 2 | 2 | 100.0 |
| Expressions | 20 | 20 | 100.0 |
| Toggles | 1600 | 1416 | 88.5 |

Figure 4 shows the MAC test in the ModelSim GUI, opened with `vsim -do gui.do`. The predefined layout (`wave_mac.do`) groups the control signals, operands and, for each of the three MAC variants, the RTL accumulator directly above the reference model, so that any divergence is visible at a glance. The product of the operands applied in one cycle appears in `p_q` one cycle later and is added to `acc` in the following cycle; while `en` is low the accumulator holds its value. The `errs` counter stays at zero for the whole run.

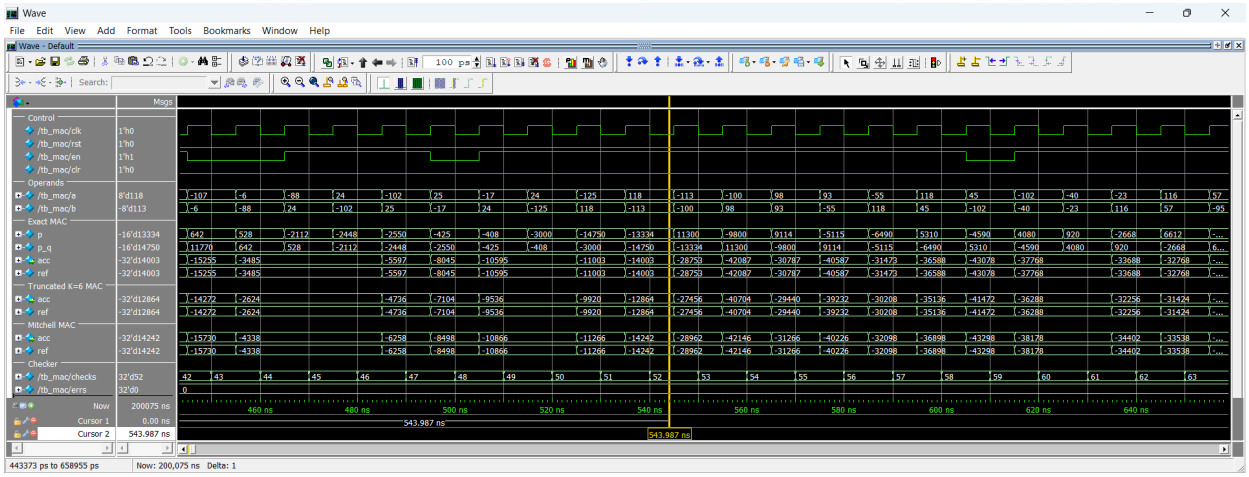



Figure 4: ModelSim SE-64 2020.4 wave window during `tb_mac`: exact, truncated ($K = 6$) and Mitchell MACs, each with its reference accumulator.

# 7 Logic Synthesis and Design-Space Exploration

## 7.1 Flow

Each multiplier and MAC was synthesized with Yosys 0.70 and ABC (`python/synth.py`). After flattening and generic optimization, the netlist was mapped by ABC onto a technology-independent library of 2-input gates (AND, NAND, OR, NOR, XOR, XNOR, AND-NOT, OR-NOT) and 2:1 multiplexers. Area is reported in NAND2-equivalent gates (GE) using relative cell areas typical of a standard-cell library (NAND2/NOR2 = 1, AND2/OR2 = 1.33, XOR2/XNOR2/MUX2 = 2.33, inverter = 0.67, flip-flop $\approx$ 4.3–6). Delay is approximated by the logic depth, i.e. the number of gates on the longest combinational path. These figures are not a substitute for a characterized PDK, but they are consistent across designs and therefore suitable for relative comparison.

For every design the flow stores the complete Yosys log, including the cell statistics and the longest path, and the gate-level netlist (`results/synth/`). Figure 5 shows a run of the flow together with the statistics that Yosys reports for the truncated multiplier with $K = 8$; the 155 logic cells match Table 5 (the additional `$scopeinfo` entry is hierarchy metadata, not logic). Figure 6 shows the MAC unit as elaborated by Yosys before technology mapping: the multiplier instance, the pipeline flip-flops (`$dff`), the accumulator adder (`$add`) and the multiplexers implementing reset,

enable and clear.

```
Yosys synthesis - approx-mac-dnn
C:\UNI\approx-mac-dnn> python python\synth.py
Yosys 0.70 + ABC, 2-input gate library

design          cells  FFs       GE  saving%  depth
exact             385    0    573.2      0.0     35
trunc_k4          314    0    465.1     18.9     33
trunc_k6          250    0    368.5     35.7     32
trunc_k8          155    0    230.7     59.8     24
mitchell          384    0    544.9      4.9     49
mac_exact         617   50   1167.2      0.0     63
mac_trunc_k6      444   44    870.3     25.4     51
mac_trunc_k8      342   42    713.5     38.9     47
mac_mitchell      621   50   1145.5      1.9     63
saved C:\UNI\approx-mac-dnn\results\synth.csv

C:\UNI\approx-mac-dnn> python python\synth_report.py trunc_k8
--- results/synth/trunc_k8.log ---
Printing statistics.
=== mul_trunc ===
      165 wires
      343 wire bits
       18 public wires
      196 public wire bits
        3 ports
       32 port bits
      156 cells
        4   $_ANDNOT_
       27   $_AND_
        1   $_MUX_
       41   $_NAND_
       12   $_NOR_
        3   $_ORNOT_
       26   $_OR_
       20   $_XNOR_
       21   $_XOR_
        1   $scopeinfo
Longest topological path in mul_trunc (length=24):

C:\UNI\approx-mac-dnn>
```

Figure 5: Synthesis flow output and Yosys cell statistics of the truncated multiplier ($K = 8$), read from its synthesis log.

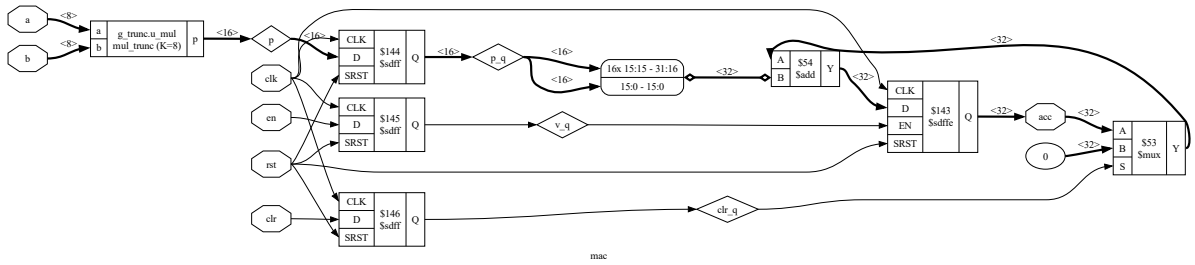



Figure 6: Yosys schematic of the MAC unit with the truncated multiplier ($K = 8$), generated with `show` after `proc` and `opt -full`. Reset and enable logic is absorbed into flip-flops with synchronous reset (`$sdff`) and enable (`$sdffe`).

Figure 7 redraws the same netlist as a register-transfer schematic. Every element corresponds to one Yosys cell of Figure 6: the multiplier instance, the four registers (`$144`–`$146` and `$143`), the clear multiplexer (`$53`) and the accumulator adder (`$54`); the sign extension is pure wiring.

## 7.2 Results

## 7.3 Discussion

**Truncation dominates the design space.** Table 5 and Figure 8 show that column truncation provides an almost linear area–error knob. Truncation at $K = 6$ removes 35.7 % of the multiplier area for a 0.04 % accuracy loss, and $K = 8$ removes 59.8 % for 0.42 %. Truncation also shortens

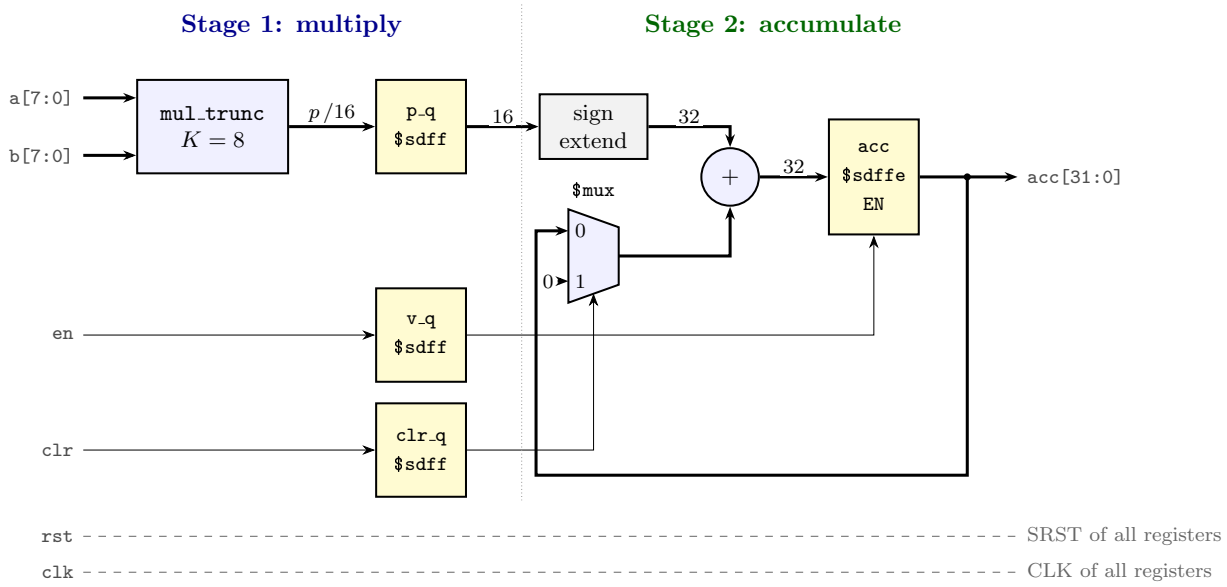



Figure 7: Register-transfer schematic of the MAC unit derived from the Yosys netlist. Thick lines are buses; `clr_q` selects the constant zero to restart accumulation, and `v_q` enables the accumulator register.

Table 5: Synthesis results. Savings are relative to the exact multiplier or the exact MAC, respectively.

| Design | Cells | FFs | Area (GE) | Saving (%) | Logic depth |
|---|---|---|---|---|---|
| Exact multiplier | 385 | 0 | 573.2 | – | 35 |
| Truncated $K$=4 | 314 | 0 | 465.1 | 18.9 | 33 |
| Truncated $K$=6 | 250 | 0 | 368.5 | 35.7 | 32 |
| Truncated $K$=8 | 155 | 0 | 230.7 | 59.8 | 24 |
| Mitchell | 384 | 0 | 544.9 | 4.9 | 49 |
| MAC, exact | 617 | 50 | 1167.2 | – | 63 |
| MAC, trunc. $K$=6 | 444 | 44 | 870.3 | 25.4 | 51 |
| MAC, trunc. $K$=8 | 342 | 42 | 713.5 | 38.9 | 47 |
| MAC, Mitchell | 621 | 50 | 1145.5 | 1.9 | 63 |

the critical path, since the removed low-order columns shorten the carry chain of the final adder. The saving carries over to the complete PE: the MAC with $K = 8$ is 38.9 % smaller than the exact MAC. Part of this comes for free, because the always-zero low bits of the product are detected by synthesis and their pipeline flip-flops are removed (50 → 42 FFs).

**Mitchell is not efficient at 8 bits.** Although its error is moderate (MRED 3.7 %), the Mitchell multiplier saves only 4.9 % of the area and has the *longest* critical path (49 vs. 35 gates). Its overhead—two leading-one detectors, two barrel shifters, a comparator and a final multiplexer—is roughly constant, whereas the partial-product array it replaces grows quadratically with the operand width. At 8 bits the array is still small, so the overhead cancels most of the benefit. Logarithmic multipliers are therefore better suited to wider datapaths (16 bits and above), while truncation-based designs are preferable for int8 DNN accelerators. This is a width-dependent conclusion that the error metrics alone would not reveal: judged only by MRED, Mitchell appears better than truncation at $K = 6$.

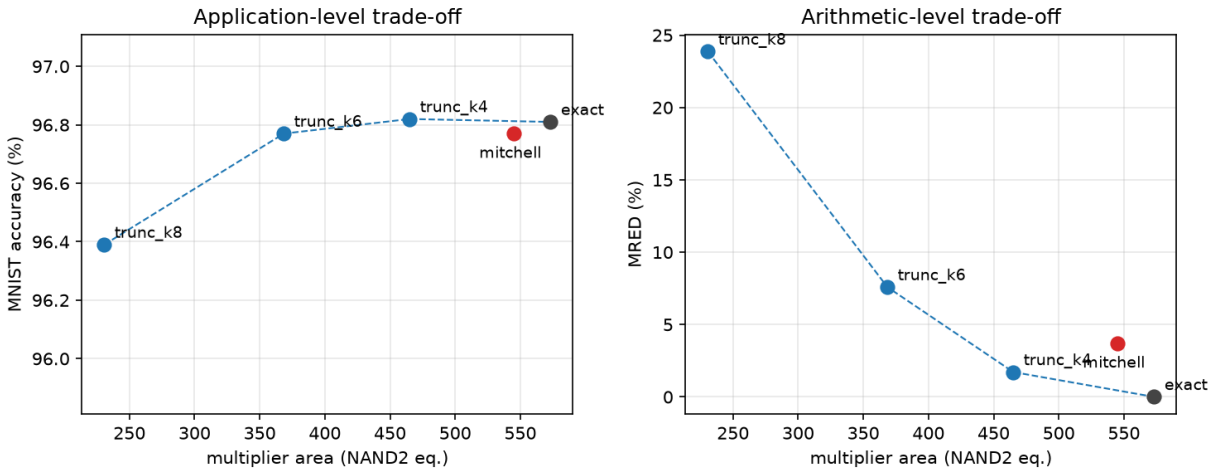



Figure 8: Design space of the 8-bit multipliers. Left: MNIST accuracy versus multiplier area. Right: mean relative error versus area. The dashed line connects the truncation family.

**Error metrics do not predict accuracy.** Truncation at $K = 6$ has twice the MRED of Mitchell but the same accuracy, and $K = 8$ has 24 % MRED with less than half a point of accuracy loss. Selecting approximate units for DNN accelerators therefore requires application-level evaluation rather than arithmetic metrics alone.

## 8 Conclusion

This work presented a complete evaluation flow for approximate arithmetic in DNN accelerators, covering bit-accurate modelling, exhaustive RTL verification, logic synthesis and quantized-network inference. For an int8 MLP on MNIST, a column-truncated multiplier with $K = 8$ reduces multiplier area by 59.8 % and MAC area by 38.9 % at a cost of 0.42 % in classification accuracy, and $K = 6$ achieves a 35.7 % multiplier area reduction with no measurable accuracy loss. The Mitchell logarithmic multiplier, despite a moderate arithmetic error, offers less than 5 % area reduction at this operand width while lengthening the critical path. The results show that the suitability of an approximate multiplier depends on the datapath width and must be judged at the application level rather than by arithmetic error metrics alone.

## 9 Repository Structure

| | |
|---|---|
| `python/approx_mult.py` | bit-accurate multiplier models |
| `python/error_analysis.py` | exhaustive error metrics and heatmaps |
| `python/train_quantize.py` | NumPy MLP training and int8 quantization |
| `python/approx_inference.py` | inference through approximate multipliers |
| `python/gen_vectors.py` | golden vectors for RTL verification |
| `python/synth.py` | Yosys/ABC synthesis, area and logic depth |
| `python/synth_report.py` | cell statistics from a synthesis log |
| `python/pareto.py` | design-space plot and summary table |
| `docs/figures/mac_unit.tex` | MAC block diagram (TikZ) |
| `rtl/` | Verilog: sign_mag, mul_exact, mul_trunc, mul_mitchell, mac |
| `tb/` | self-checking testbenches, run_sim.sh |
| `sim/modelsim/` | ModelSim project, do-scripts, coverage flow |
| `results/` | metrics (CSV), figures, quantized model |
| `docs/report.tex` | this report |

All experiments are reproducible with `pip install -r requirements.txt`, running the Python scripts in the order listed above, and `vsim -c -do regression.do` in `sim/modelsim` for the RTL regression with coverage (`sh tb/run_sim.sh` provides a minimal flow that also works with Icarus Verilog).

## References


[1] J. N. Mitchell, "Computer multiplication and division using binary logarithms," *IRE Transactions on Electronic Computers*, vol. EC-11, no. 4, pp. 512–517, 1962.

[2] H. Jiang, F. J. H. Santiago, H. Mo, L. Liu, and J. Han, "Approximate arithmetic circuits: A survey, characterization, and recent applications," *Proceedings of the IEEE*, vol. 108, no. 12, pp. 2108–2135, 2020.

[3] V. Mrazek, R. Hrbacek, Z. Vasicek, and L. Sekanina, "EvoApprox8b: Library of approximate adders and multipliers for circuit design and benchmarking of approximation methods," in *Proc. DATE*, 2017, pp. 258–261.

[4] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. CVPR*, 2018, pp. 2704–2713.

[5] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.